

RAIDTEST 第三版技術報告

1. 專案功能現況總覽 (Project Capability Map)

- **非對稱 RAID0 功能**：已實作可處理多顆速度不同的 SSD（如 NVMe + SATA 混搭），依據各碟實際 I/O 效能比重分配條帶大小（如 README 所述，透過「快碟擔更多、慢碟擔較少」達成）【20†L182-L190】。資料分布方式由 *stripe_engine* 模組動態計算所有分區相對起始位置及大小，保證每筆寫入在所有磁碟「同時完成」時即結束，避免傳統 RAID0 因最慢磁碟拉低效能的問題。
- **磁碟效能測試 (Scan 測速)**：在 `bench_io.c` 中以高效能寫入檔案，使用 `CreateFile` 搭配 `FILE_FLAG_NO_BUFFERING`（相當於類 Unix `O_DIRECT`）和疊代式覆寫檔案以測量實寫效能。未使用系統快取可直接反映硬體速度，但也會失去作業系統的預讀優化【20†L182-L190】。實驗表明對於順序讀寫，繞過快取可能明顯降低效能，但對於大檔案或資料庫複製場景則可避免快取干擾。如有需求，可加上 `FILE_FLAG_SEQUENTIAL_SCAN` 提示以改進效能【20†L326-L334】。
- **非對稱動態條帶路由**：核心實現於 *stripe_engine*。根據檢測出各磁碟之實際寫入速度 (`bench_write_mbs`) 計算相對比率，再動態將可用容量分為多個「階段」(Phase)。每個 Phase 定義有：參與的磁碟數量、其速度比率總和，以及此階段可提供的虛擬區塊大小。系統依序判斷速度最慢磁碟何時用盡空間，減少活躍磁碟數（類似逐步淘汰），對應產生多個 Phase。查詢時，依據虛擬區段偏移量可映射到該 Phase 內各磁碟的實體位址與長度，由此完成 LBA-to-Physical 的動態映射。
- **4GB RAM 快取 (Dirty Page Cache)**：在 Volume 內配置一塊固定 4GB (或設定值) 的記憶體快取區塊，將連續位址映射到 `VirtualAlloc` 出來的緩衝區。任何寫入優先寫入此快取（寫回模式），後台另有兩階段 Flush 機制：① 每次呼叫 `fsync/flush` 時立即同步寫出 (`cache_flush_all`)；② 背景執行緒定期執行 Flush，根據髒頁比率調整時間間隔 (`dirty`>50% 時 100ms, >25% 時 250ms, >10% 時 500ms, 預設 1000ms)。此寫入快取由 Windows `CRITICAL_SECTION` 保護臨界區，確保多執行緒下對 `cache->buffer` 與髒頁位圖的存取互斥安全，避免競爭條件和位圖損毀。
- **WinFsp/FUSE 固定磁碟掛載**：利用 WinFsp 提供的 FUSE 介面，將 RAIDTEST 卷掛載為 Windows 本地固定磁碟 (如 `G:`)。核心實作在 *fuse_bridge.c*，定義了對應的 FUSE callback (如 `getattr`、`readdir`、`read`、`write`、`truncate`、`flush`、`fsync`、`statfs` 等)，並使用 `fuse_mount` 建立多執行緒 FUSE Loop。此方式讓使用者空間應用無需 NTFS 驅動知識，即可透過標準 API 存取虛擬磁碟。WinFsp 程式架構證實穩定可用，且已在 README 中說明與 Linux FUSE 的相似概念【15†L299-L307】。
- **JSON 狀態持久化與載入**：在 *config.c* 中以 JSON 文字格式儲存全域設定 `config.json`，包括 RAID 版本、快取大小 (`cache_mb`)、掛載磁碟代號等。保存時使用 `fwprintf` 按欄位輸出，各磁碟以 `{ "id": ..., "drive": "X", "pool_mb": ... }` 列出。讀入時使用 `swscanf` 解析每行 JSON 片段，將參數回寫至 `APP_CONFIG` 結構。此方式可讓重新啟動後恢復先前配置 (Persistence Cycle)，在讀取後主程式自動重新以相同參數重建檔案系統結構。

- **GUI 視窗及控制緒同步**：以 Win32 GUI 專案實作使用者介面，包括按鈕觸發配置精靈、掛載等動作，以及動態顯示日誌欄和狀態列表。由於後台作業（掃描磁碟、效能測試、格式化等）在獨立緒執行，使用 `PostMessage / SendMessage` 向主執行緒更新 UI 元件（如 ListView），確保線程間同步且避免直接跨緒操作視窗。例如將 `<LB_ADDSTRING>`、`<LVM_INSERTITEM>` 等訊息傳送給 GUI 控制項，避免競爭條件或崩潰。

2. 核心架構與實作方法解析 (Implementation Architecture)

- **Stripe Engine (條帶引擎)**：初始化時由 `stripe_volume_create` 函式建立虛擬卷 (`STRIPE_VOLUME`)，計算每磁碟相對速度與分配比率。先將所有磁碟視為活躍 (`remaining_disk_count`)，累加速率求得總比率；再以迴圈方式，計算此 Phase 的迴圈大小 (`cycle_size = (sum of ratios) * stripe_unit`)。對每磁碟計算可用虛擬空間上限：
$$\text{max_virtual_i} = (\text{remaining_phys_bytes_i} * \text{total_ratio}) / \text{ratio_i}$$
。找出最小者作為此 Phase 可用的迴圈數
$$\text{cycles} = \text{floor}(\min(\text{max_virtual}) / \text{cycle_size})$$
，並設定
$$\text{virtual_size} = \text{cycles} * \text{cycle_size}$$
。所用實體空間則等於對應速率份額乘以迴圈次數。完成後移除空間用盡的慢速磁碟，重複分割剩餘磁碟，直到只剩單碟或無空間。如有單碟剩下，直接把剩餘容量投入最後 Phase。最終，每個 Phase 物件保存：參與磁碟列表、各自比率、總比率、虛擬起始位址與大小等。實際 I/O 時的 LBA 映射 (`stripe_volume_map_lba`) 程式碼會搜尋對應 Phase，依偏移算出屬於哪些磁碟的區塊，每磁碟讀寫起始位址和長度，交由 `stripe_read_raw / stripe_write_raw` 執行低階 I/O。
- **Ram Cache (快取子系統)**：以 `cache_init` 配置一段連續 4GB (或設定值) 虛擬記憶體，並建立對應髒位圖 `dirty_map` (位元組陣列，每位表示 4KB Block 是否髒)【21†L98-L106】。寫入時，`cache_write` 先以 `EnterCriticalSection` 上鎖、將資料 `memcpy` 到快取緩衝區，標記覆蓋區塊為髒；然後釋放鎖。讀取快取有髒處直接回傳、否則依照映射讀取實體磁碟。背景有一個專用 flush 執行緒 (`cache_flush_thread`)，反覆鎖住位圖檢查髒塊併成批 (最多 256 塊連續髒塊) 執行 `WriteFile` 重疊寫入到相應磁碟位置，並在失敗時重新置位髒位圖【21†L98-L106】。`raid_flush` 與 `raid_fsync` 回調亦會主動呼叫 `cache_flush_all`，確保必要時強制同步。在同步過程釋放髒位後，快取區塊即被視作已提交。此設計利用兩階段緩衝，兼顧延遲寫入效能與資料可靠性（必要時可加 `FlushFileBuffers` 強制真正落盤）。
- **Pool I/O (磁碟池檔案管理)**：每顆實體磁碟上建立一個儲存池檔案 (Pool file)，大多以未加快取 (`FILE_FLAG_NO_BUFFERING`) 模式開啟，並執行等同容量設定。`pool_read / pool_write` 定位 Pool 檔案對應位置後以覆蓋方式讀寫原始區塊。寫入時在每次 `WriteFile` 後呼叫 `FlushFileBuffers` 強制將資料推到磁碟，避免 Windows 快取殘留導致資料落盤延遲。【21†L98-L106】指出，頻繁使用 `FlushFileBuffers` 會嚴重影響效能，因此實際程式只在寫入完整區塊後 flush；在一般 I/O 路徑（如檔案寫入）則依需要觸發。在關閉池檔前亦會呼叫 `FlushFileBuffers` 確保完整落盤。
- **Superblock (元資料結構)**：為描述整體 RAID 結構，每個磁碟的根目錄下保有「RAIDTEST.SUPERBLOCK」檔案，由 `superblock_write` 寫入。此結構包括：魔術數 `magic`、版本號 `version`、磁碟數、每 Phase 數量、虛擬總容量等，並對整體 `STRIPE_VOLUME` 和每磁碟所屬區域計算 CRC32 校驗碼。寫入時先建立臨時檔寫入結構，再以 `MoveFileExW` 原子覆蓋正式檔。讀取 (`superblock_read`) 時打開檔案檢查長度與 CRC，並復原 `STRIPE_VOLUME` 中的所有設定 (包括多 Phase、磁碟 Pool 大小、寫入速度

等)。讀取後立即回放 Phase 配置，準備好隨後處理 I/O；因此 JSON 與二進位 Superblock 共同實現「關機-開機」後恢復整體卷狀態的流程。

- **GUI 及命令處理：**GUI 採多緒設計，一個緒專職處理 Win32 介面事件，其他緒執行磁碟掃描、效能測試、格式化等操作。所有耗時任務均透過訊息傳遞 (PostMessage) 通知介面緒更新狀態。例如執行磁碟掃描完畢會在主執行緒的 ListBox 顯示裝置資訊；Mount 操作完成會透過 WM_USER_DONE 通知。內部使用 gs_lock/unlock 全域鎖控狀態共享資料，避免多緒同時修改 g_state 或 UI 造成亂序。若以 CLI 模式 (參數開頭) 執行，則呼叫 cmd_handler.c 處理指令；否則進入 gui_run() 開啟視窗。

3. 系統隱患、死角與安全漏洞掃描 (Vulnerability & Weakness Scan)

- **statfs 空間報告滯後 (statfs / truncate)：**目前 raid_statfs 利用 g_ctx.highest_byte_written 估算已用區塊數 (used_blocks = highest_byte_written / SECTOR_SIZE + 1) 再計算剩餘空間，但 highest_byte_written 只隨寫入增加從不減少，故檔案縮減 (truncate) 後並不更新此值，statfs 會顯示過多已用空間而不反映釋放情況【13†L226-L229】。根據統計，struct statvfs 的 f_bfree 和 f_bavail 欄位必須正確設定，否則工具例如 df 將顯示錯誤可用空間【13†L226-L229】。此設計缺陷在高更新或多檔操作下易累積誤差，導致使用者空間或應用程式無法正確取得剩餘容量資訊。
- **高併發寫入下快取溢位風險：**在大量並行寫入時，雖使用 EnterCriticalSection 保護快取緩衝，但遇極端情況仍有可能競爭或耗盡資源。例如多緒頻繁操作同一區塊或緊鄰區塊時，臨界區鎖爭可能變成效能瓶頸。此外，當 cache_write 中的 offset+length 超出配置大小時才會阻擋，但如果程式錯誤傳入較大值，memcpy 也可能越界；目前僅檢查 end > cache_size，但未檢查與到達頁尾的位對齊 (Windows NO_BUFFERING 要求)，若無對齊措施可能觸發異常。此外，快取寫入後發起多執行緒背景 WriteFile 時，取消或重試邏輯複雜；若未妥善處理返回值，將出現 I/O 競爭狀況或資源洩漏。
- **Superblock 容錯不足：**設計上將整個卷的元資料同步寫入每個磁碟上的同名檔案 (RAIDTEST.SUPERBLOCK)。雖然檢查 CRC 保證單一檔案有效性，但實際讀取時只打開了其中一個 (未具備多重備份機制)；若該磁碟損壞，整個卷結構無法還原。且 RAID0 本身無冗餘，任何單一磁碟失效即資料喪失。建議一般 RAID0 需考慮磁碟故障策略，目前設計對單碟故障風險未做偵測 (如未定期驗證校驗碼或持續性檢查)。若寫入途中忽略錯誤，將導致超出卷長度或不同磁碟不一致問題。缺少跨磁碟的校驗或日誌機制 (Journaling)，使得突然斷電或軟體異常關閉時，卷結構可能出現不一致而難以修復。
- **Daemon 模式退出不優雅：**以 daemon.c 實現的守護模式，在控制台接收到 Ctrl+C 或輸入 exit 時會呼叫 cleanup_session 與 cleanup_disks 做善後。但現行代碼在某些情況下可能跳過最後資源釋放。例如 WaitForMultipleObjects 只檢查標準輸入和停止事件，未監聽其他系統信號 (如服務停止信號)；若程式被強制終止，無法呼叫 FlushFileBuffers 等確保資料落盤。此外，CloseHandle、記憶體釋放等動作並非全覆蓋 (如未關閉所有檔案 handle)，可能造成資源洩漏。依據寫入快取與 FUSE 設計，若背景 Flush 執行緒未妥善終止，關閉 Volume 前容易遺留部分髒頁未寫出。整體而言，缺少更細緻的信號處理與等待緒終止機制，若現場崩潰 (如關閉主視窗) 可能導致檔案系統狀態遺失或不一致。
- **其他效能／一致性考量：**大量使用 FlushFileBuffers 可確保資料完整，但正如文獻所述，其會嚴重降低系統整體吞吐【21†L121-L129】。目前程式在格式化或創建新檔時多次呼叫。若需同時兼顧性能與資料安全，可研究減少 Flush 次數、改為使用 FILE_FLAG_WRITE_THROUGH 或主動在安全點時一次性 flush。亦需注意 FILE_FLAG_NO_BUFFERING 的特殊需求 (資料需 Sector 大小對齊)【21†L25-

L34]，否則可能在某些硬體上出錯。另外，`statfs` 實作參考亦顯示需設定 `f_bfree`、`f_bavail` 等欄位【13†L226-L234】，目前僅設定了 `f_blocks`、`f_bfree`、`f_bavail`，尚可再檢討 `f_files`、`f_ffree` 是否符合檔案數限制規範。

4. 具體的代碼改進與修改方案 (Refactoring & Action Plan)

針對上述每項隱患，提出改進方向和範例代碼：

- **修正 statfs/空間計算**：修改 `raid_truncate` 或檔案關閉邏輯，使刪除或截斷時更新空間計算。具體可以在 `raid_truncate` 中追蹤實際檔案大小變更，例如：

```
static int raid_truncate(const char* path, fuse_off_t size) {
    if (path[0] == '/') {
        int idx = find_open_file(path + 1);
        if (idx >= 0) {
            // 更新檔案大小
            uint64_t old_size = g_open_files[idx].size;
            g_open_files[idx].size = (uint64_t)size;
            // 調整最高已寫位址，確保不超過當前檔案新長度
            if (g_ctx.highest_byte_written > (uint64_t)size) {
                g_ctx.highest_byte_written = (uint64_t)size;
            }
            // 或追蹤釋放空間量以供 statfs 使用
            g_ctx.total_bytes_freed += (old_size - (uint64_t)size);
        }
    }
    return 0;
}
```

同時在 `raid_statfs` 計算時應考慮已釋放的空間：

```
uint64_t used_bytes = g_ctx.highest_byte_written - g_ctx.total_bytes_freed;
st->f_bfree = st->f_blocks - (fuse_fsblkcnt_t)(used_bytes / SECTOR_SIZE);
st->f_bavail = st->f_bfree;
```

如此當檔案縮小時可以恢復對應空間，避免 `statfs` 過度報滿。也可改為累積所有已開檔案的大小和關閉時減少，而不是只用最高值。

- **加強快取寫入安全性**：對於快取緩衝與隣頁位圖操作，建議改為 C++ `std::mutex` 或 `std::lock_guard`，提高可讀性與安全性；或使用 `atomic` 位元組陣列操作，以避免多緒下的微弱競爭。例如：

```
class RamCache {
    std::vector<uint8_t> buffer;
```

```

std::vector<std::atomic<uint8_t>> dirty_map; // 每位元原子操作
std::mutex mtx;
public:
    bool write(uint64_t offset, const void* data, uint32_t len) {
        if (!data || offset + len > buffer.size()) return false;
        std::lock_guard<std::mutex> lock(mtx);
        memcpy(buffer.data()+offset, data, len);
        for(uint64_t b = offset/CACHE_BLOCK_SIZE; b < (offset+len +
        CACHE_BLOCK_SIZE-1)/CACHE_BLOCK_SIZE; b++){
            dirty_map[b] |= (1 << (b % 8));
        }
        return true;
    }
    //...
};

```

此設計避免裸指標使用與手動位元操作的易錯；同時可進一步檢查齊位要求，確保在 `NO_BUFFERING` 模式下 I/O 長度與位移符合硬體需求，避免跨越邊界時的錯誤。背景寫入也可考慮 `std::thread` 取代 `_beginthreadex`，並在退出時用標誌與 `join()` 確保緒正常結束。

- **改善 Superblock 容錯：**在 `superblock_read` 除了驗證單一檔案外，可嘗試對所有磁碟上的 Superblock 做讀取與 CRC 檢查，選擇最完整或版本最高的一組載入。示意：

```

bool load_superblock_from_any(STRIPE_VOLUME* vol) {
    for (int i = 0; i < vol->disk_count; i++) {
        wchar_t drv[4] = { vol->disks[i]->drive_letter[0], L':', L'\\',
        0 };
        if (superblock_read(drv, vol, ..., &disk_count_out)) {
            LOG_OK("Loaded superblock from disk %c", vol->disks[i]-
            >drive_letter);
            return true;
        }
    }
    LOG_ERROR("All superblocks invalid");
    return false;
}

```

或者在寫入時引入「主次備份」機制，例如同時寫入主 Superblock 並保留過去 n 版快照，並將其鏡像存放在其他磁碟。如此若主寫失敗，可自備用檔案還原。此外，可考慮在程式啟動與運作時定期驗證 Superblock CRC，提早偵測腐壞並提示警告。

- **增強守護程式退出控制：**在 `daemon_main` 循環外添加更多事件監控，例如監聽 Windows 服務停止事件或進程終止通知，並在退出前確保所有檔案系統操作完成。可使用 `SetConsoleCtrlHandler` 以外，搭配 `RegisterServiceCtrlHandler` 處理服務模式下的停止信號。退出時，先 `WaitForSingleObject(flush_thread, INFINITE)`，確保快取寫入緒完成，再依序呼叫

`fuse_unmount_volume` 和所有關閉函式。例如：

```
// 設定 Service 控制回撥
SERVICE_TABLE_ENTRY ste[] = { {SERVICE_NAME,
(LPSERVICE_MAIN_FUNCTION)ServiceMain}, {NULL, NULL} };
StartServiceCtrlDispatcher(ste);
// 在 ServiceMain 中:
SetServiceStatus(..., SERVICE_STOPPED, ...);
// ServiceStop Handler:
g_ctx.vol->cache.running = 0;
WaitForSingleObject(flush_thread, 5000); // 等待快取寫入
fuse_unmount_volume(...);
cleanup_session(...); cleanup_disks(...);
```

並於 `cleanup_cache` 中延長等待，避免 3 秒後強制關閉導致未寫出的脏页遺失。此舉可確保資料完整性與記憶體正確釋放。

- **優化 I/O 與快取策略：**綜合效能與安全考量，可在創檔和寫入時選用 `FILE_FLAG_WRITE_THROUGH`，讓系統在不禁用快取的情況下立即將資料落盤【21†L50-L59】。同時避免過度頻繁呼叫 `FlushFileBuffers`，改為批次寫入結束後一次性 flush 或在適當時機觸發，減少對效能的負面影響【21†L121-L129】。在需要快速掃描大檔案時，可在 `CreateFile` 加上 `FILE_FLAG_SEQUENTIAL_SCAN` 提示，配合適當的讀取緩衝佈局以充分利用 OS 快取預讀功能【20†L326-L334】。
- **其他細節：**對 CLI/GUI 部分，可將全域 `g_state` 改為物件或單例模式包裝，減少跨函式作用域的共享，並加強錯誤回傳檢查。例如，加入對 `CONFIG_DIR` 與 `SUPERBLOCK_FILENAME` 是否一致的校驗。對 JSON 讀寫部分，可考慮使用專門的 JSON 庫（如 [nlohmann/json](#)）提高序列化/反序列化的安全性和可維護性。今後開發中，以上建議會成為 Phase 4.2 的重構重點，以補強健壯性和維持高效能。

參考資料：WinFsp FUSE 描述【15†L299-L307】、`FILE_FLAG_NO_BUFFERING` 性能影響【20†L182-L190】【20†L326-L334】、`statvfs` 字段用法【13†L226-L234】、`FlushFileBuffers` 性能影響【21†L121-L129】等。